Operating Systems Assignment

Ruth-ann Rudolph
RDLRUT001

Due: 05/05/2025

—

CSC3002F

—

Report document

Optimal Time Quantum:

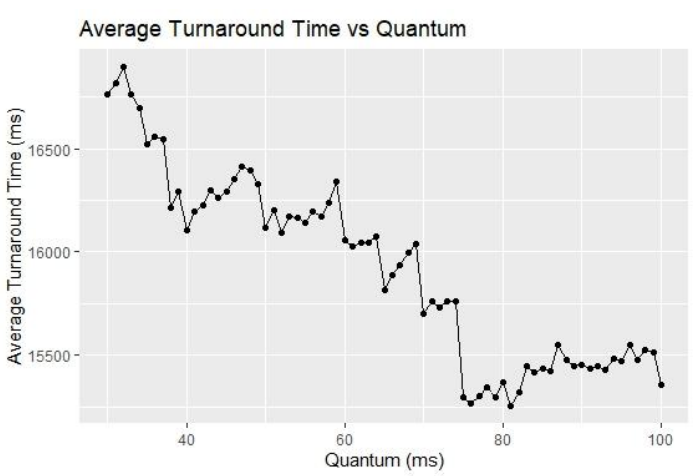
Initial planning:

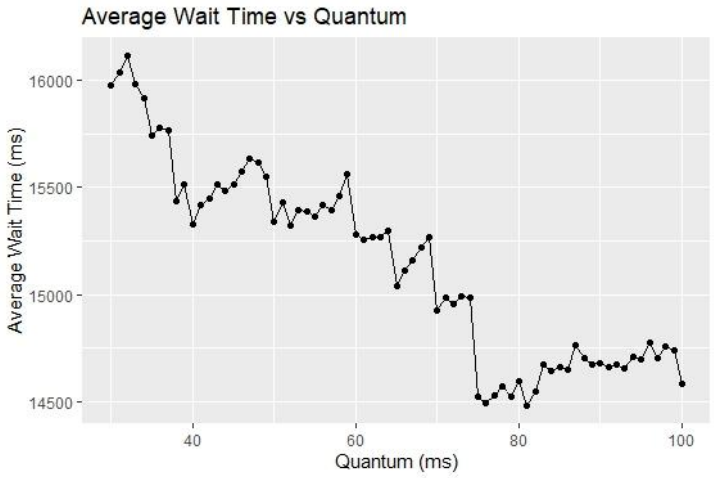
Taking all drink preparation times as CPU bursts:

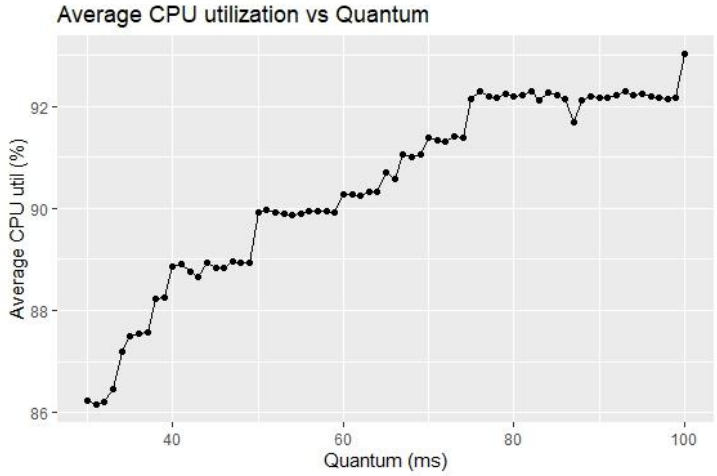
- Average time: 60 seconds (milliseconds in simulated time)
- Median time: 50 seconds (milliseconds in simulated time)
- Range of drink preparation times: 3 to 200 seconds

Therefore, in keeping with the standard of having the time quantum be large enough that 80% of the CPU bursts complete in a single time quantum, the predicted ideal time quantum will be approximately 75 seconds (milliseconds in simulated time).

To test this, several simulations were conducted, for 25, 50, 75 and then 100 patrons. Each time the Round Robin algorithm was tested using a range of seeds to simulate repeated tests under differing conditions (14, 25, 36, 47, 58, 69, 70). To ensure a wide enough range of time quanta, each test was run for 30 to 100 seconds (milliseconds). The results were then averaged over all tests for each time quantum and produced the following results:







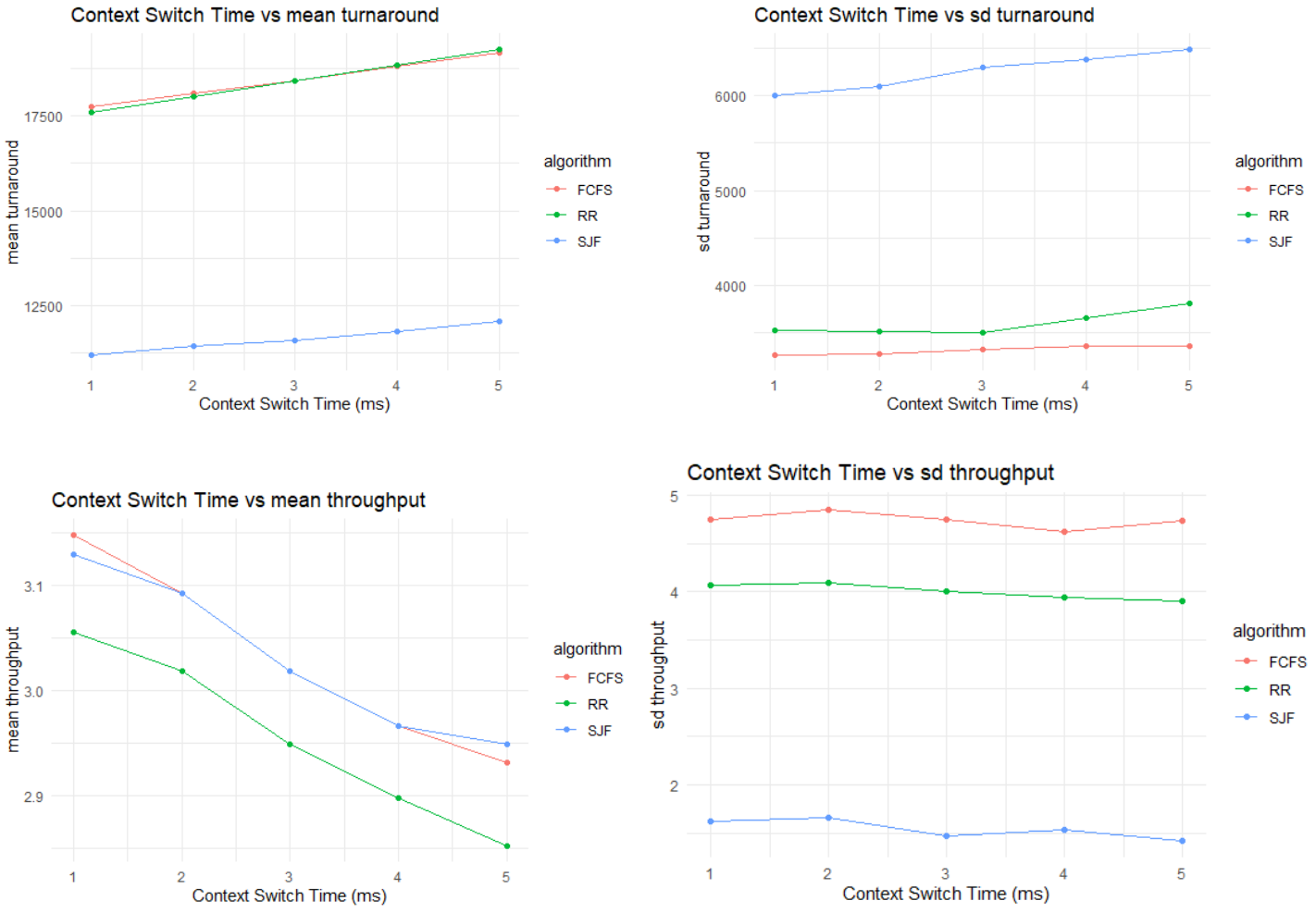
These results support the initial assumption that having a time quantum large enough that 80% of the CPU bursts complete within it give the ideal time quantum, as from 75 onwards the graphs flatten dramatically. However, taking the exact lowest value gives an ideal time quantum of 81 milliseconds for the average turnaround times and wait times, however the CPU utilization spikes at 100ms, but this is not outweighed by the other ideal times. The data also show that the standard deviation

decreases from 30 to 60 and thereafter flattens and thus does not suggest any other factors should be considered. Therefore 81 seconds (milliseconds in simulated time) is selected as the ideal time quantum.

Context switch testing:

Given that larger context switching times produce larger overhead, it is expected that for every statistic, performance will decrease with larger context switching times. However, the context switch cannot simply be set to zero, as there will naturally be some overhead in both the real-world scenario of the barman and the real context of an operating system, and thus the statistics have been summarized and plotted as follows:
For 25, 50, 75 and then 100 patrons. Each time every algorithm was tested using a range of seeds to simulate repeated tests under differing conditions (14, 25, 36, 47, 58, 69, 70). Over the range of 1 to 5 seconds (milliseconds). The results were then averaged over all tests for each context switch, for each algorithm and produced the following results (excluding those which are unnecessary):

- The CPU utilization decreased linearly for each algorithm with increasing context switch.
- Response and wait times follow the same pattern as that shown in the turnaround time.



From this, one can see that all 3 algorithms behave as expected and thus a balance between ideal and real-world scenario was chosen. And the final context switch for the last set of tests is set to 3.

Overall Algorithm Comparison:

For the final comparison of all the algorithms, the quantum for Round Robin is set to 81 and the context switch for all 3 algorithms set to 3. The tests were conducted over the same set of seed values (14, 25, 36, 47, 58, 69, 70) over 5, 10, 25, 50 and 75 patrons. To ensure accurate readings, the tests were repeated 4 times and finally the average values for each statistic (mean, median and standard deviation) recorded and plotted over the number of patrons.

CPU Utilization:

- Definition: The ratio of time the CPU is busy to the time it is idle
- Description of recordings: For this simulation, a tally of time the barman spends making drinks and the time the barman spends idle (waiting for drinks or context switches) was kept throughout each run, at the end of the simulation the percentage CPU utilization was calculated as CPU_Busy_Time/(CPU_Busy_Time + CPU_Idle_time) *100. Producing the following graph:

Throughput:

- Definition: The number of processes that complete their execution per time unit
- Description of recordings: A process in this simulation is considered to be a patron, thus, the number of patrons who complete all 5 drinks per unit of time represents throughput. To record this, the manager class records the start and end time of the simulation, while each patron records and reports the time at which it finishes its execution (because all is done using the system clock, this is accurate) once all processes are finished, the manager class determines in which window the process completed, and thus has an array of windows, each containing the number of processes which completed during such a window. The mean, median and standard deviation were then computed and used to produce graphs. The Median vs mean gives an indication of whether the beginning was very unproductive and the standard deviation gives an indication of how much variation existed within each run.

Turnaround time:

- Definition: The interval from the time of submission of a process to the time of completion, in other words, the total amount of time to execute a particular process
- Description of recordings: Each individual patron records the time at which it begins (submits its first drink order) and the time at which it completes (finishes its final drink order). Then computes its total execution time (finish – start). At the end of all the processes, the manager class complies all the patron times and calculates the mean, median and standard deviation. Once again the standard deviation gives an indication of the variation, which in this case can be used to indicate fairness, starvation and predictability.

Waiting time:

- Definition: The total amount of time a process has been waiting in the ready queue.
- Description of recordings: The wait time is calculated per patron and is done in the barman class because only the barman knows which drink is being prepared and which drinks are still in the queue. When a drink enters the queue a second, parallel, queue records the time at which it enters the queue, this is then used to calculate the total waiting time for each patron,

over all 5 drink orders. Note that if an order re-enters the queue during Round Robin processing, the additional time in the queue is also added to the total wait time.

Response time:

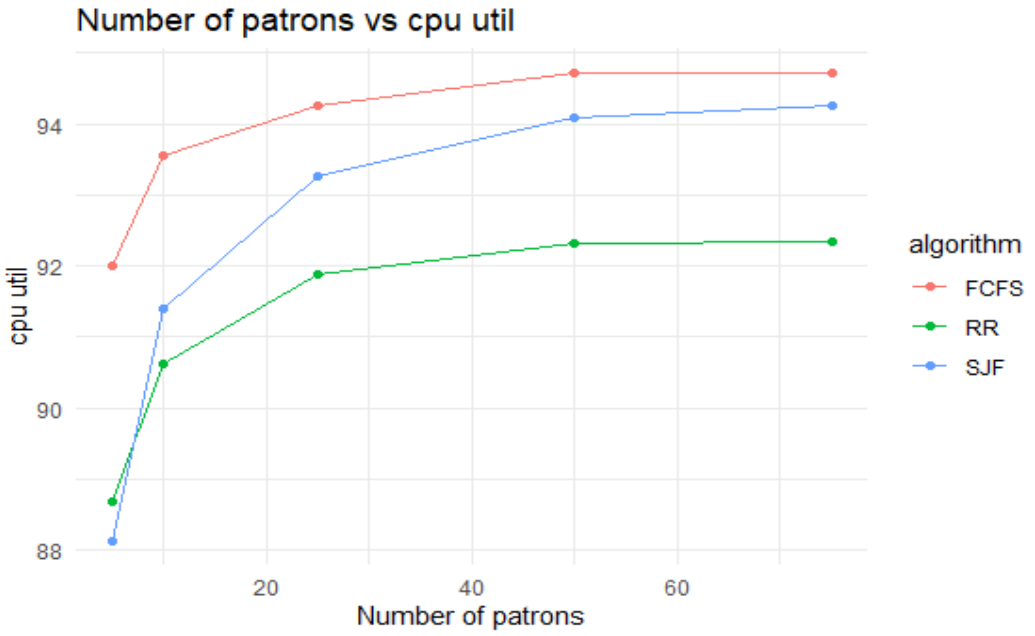
- Definition: The amount of time from when a request is submitted until the first response is produced.
- Description of recordings: Response time per patron in this case is the time from when the first drink order is placed to when the first drink is received. Each patron records its own response time, using the time from its first submission to the time of receiving its drink. At the end of all the processes, the manager class complies all the patron times and calculates the mean, median and standard deviation. Once again the standard deviation gives an indication of the variation, which in this case can be used to indicate fairness, starvation and predictability.

Other Metrics:

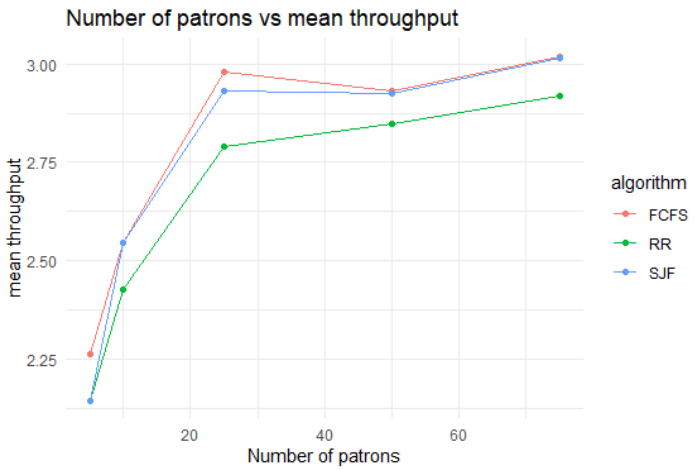
- Predictability: using the standard deviation – as a large standard deviation indicates unpredictable outcomes.
- Fairness – fair sharing of resources and overall lack of starvation, we expect first come first served and round robin to be the most fair.
- Lack of starvation – using the difference between the medians and means as an indicator. A median which is substantially lower than the mean indicates that there is an outlying large value, which would indicate starvation.

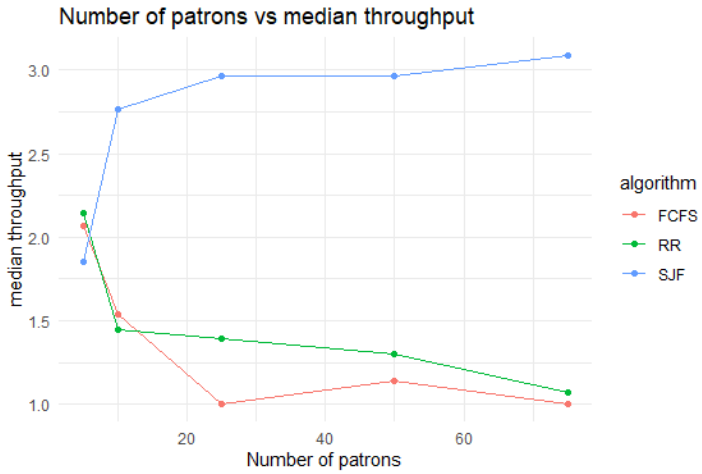
Graphs:

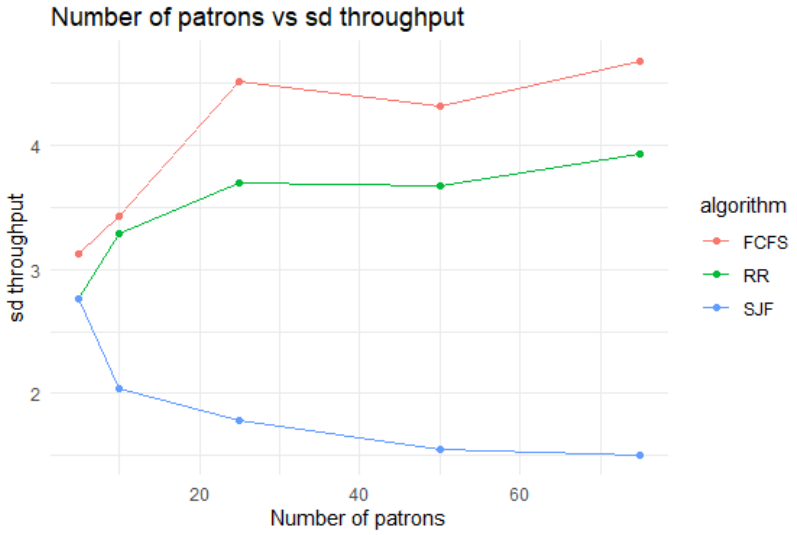
CPU Utilization:



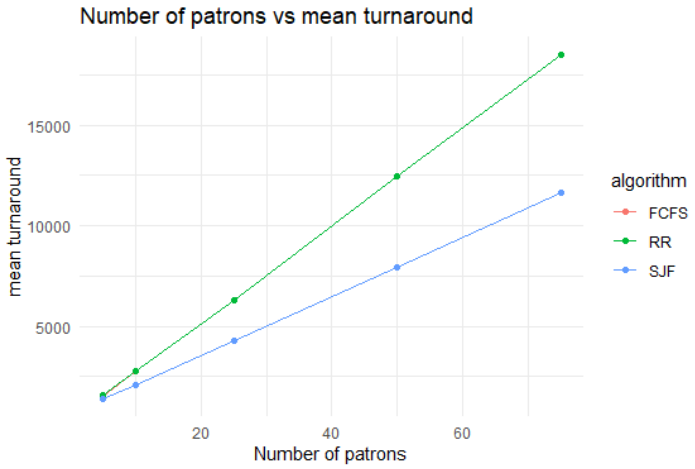
Throughput:

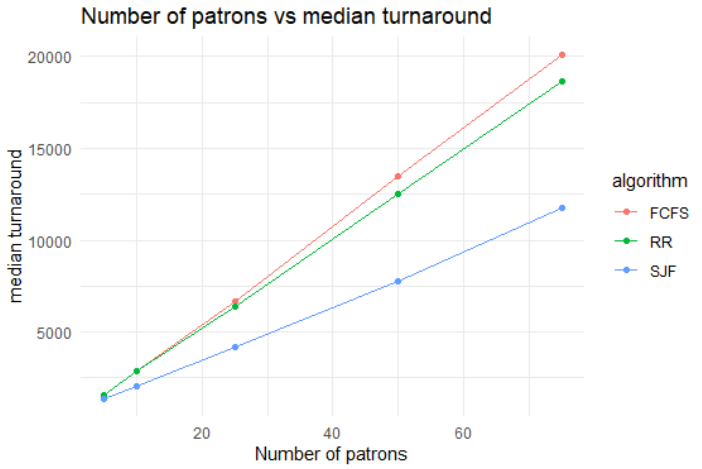


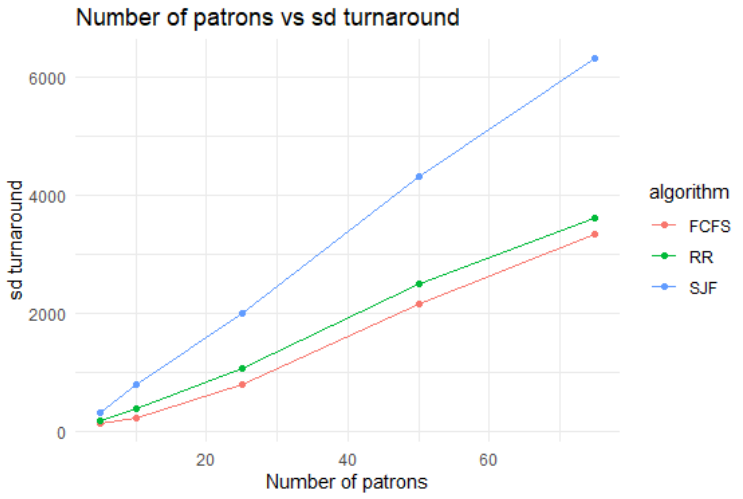




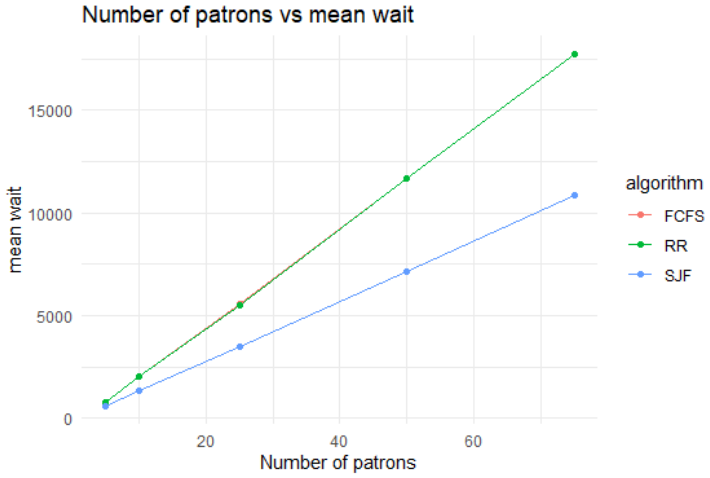
Turnaround Time:

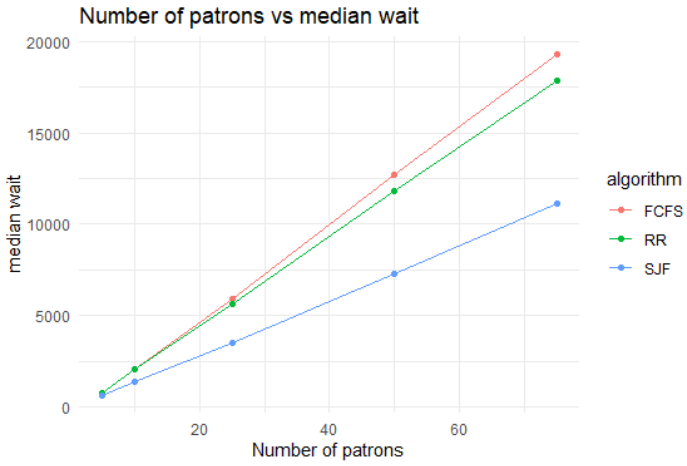


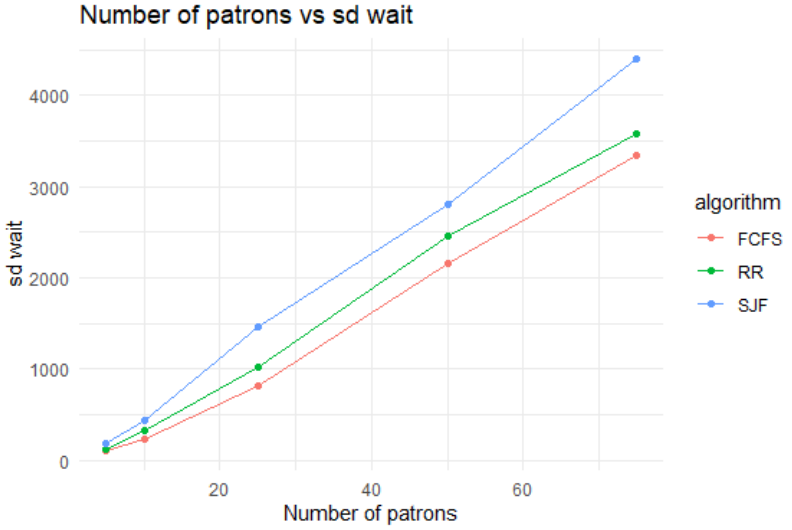




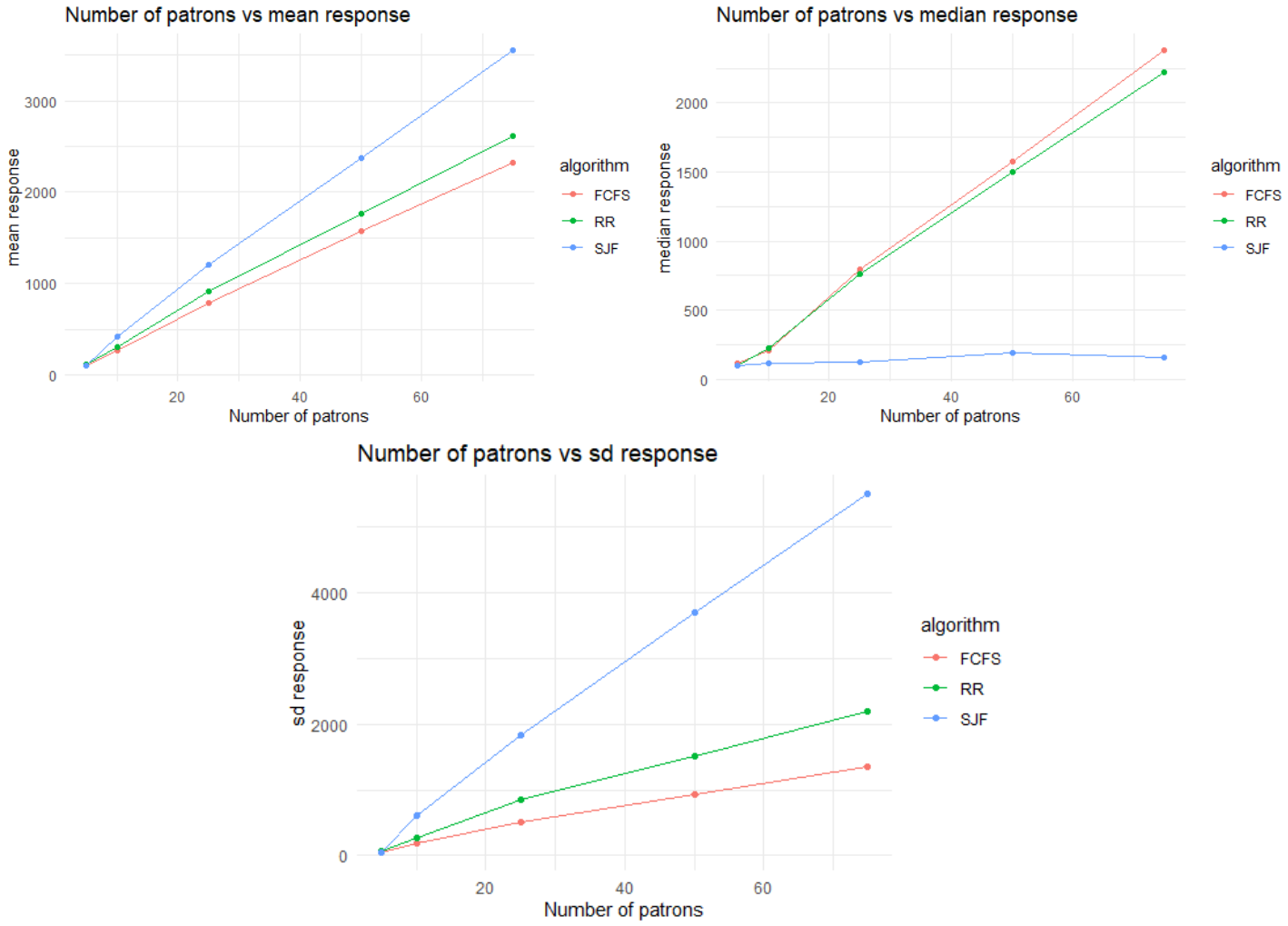
Waiting time:







Response time:



Interpretation Of Results:

Shortest Job First:

- CPU utilization:
  - Initially, the CPU utilization is lowest but quickly increases to be higher than round robin but is consistently lower than first come first served.
  - This is as we would expect due to the overhead involved in finding the shortest job.
- Throughput:
  - Mean: Roughly the same as first come first served, being the two highest. Overall, the throughput average increases with number of patrons as we would expect.
  - Median: Roughly the same as for the mean, notably – this is the only algorithm for which mean and median throughput are roughly the same, indicating that it has the least number of zeros initially and thus the most consistent throughput.
  - Standard deviation: As with the median, the overall low standard deviation indicates that the algorithm is fairly consistent. The deviation also decreases with increasing number of patrons, likely due to the ability to better schedule the shorter jobs first due to there being more short jobs.
- Turnaround time:

    - Mean and median: Increases linearly with the number of patrons, but remains the lowest over any number of patrons, mean and median being nearly the same indicate that there are neither excessive number of low times nor high times individually (however there remains the possibility that both high and low values exist).
    - Standard deviation: Also increases linearly with number of patrons, however, the standard deviation for SJF is consistently higher than the other algorithms. This is as we would expect, however, it does pose a potential problem of starvation for some patrons, as well as being a concern that it has the lowest averages but the highest standard deviation.
- Wait time:
    - Mean and median: Increase linearly with the number of patrons, but as we would expect, SJF maintains the lowest values by some margin overall.
    - Standard deviation: As with turnaround time, SJF has the highest overall standard deviation for wait time, which once again is consistent with expectations, however is never excessively higher than the other algorithms, maintaining only a few hundred ms above the others, which would translate to a couple of minutes at most in the scenario.
- Response time:
    - Mean: SJF has the highest overall mean, once again increasing linearly with the number of patrons, this is likely explained by some patrons having very fast response times and others having excessively long times, as will be seen in the following statistics.
    - Median: The median remains almost perfectly flat over any number of patrons, while this initially seems surprising, it is consistent with what we would expect due to many patrons having exceedingly fast response times, and only a few having excessively long response times (those who placed orders which take a long time to prepare as their first orders)
    - Standard deviation: Substantially higher for SJF than any other algorithm, this is exactly as one would expect due to some patrons having very low response times and others having extremely high response times. This is an indication of starvation.

First Come First Serve:

- CPU utilization: Highest overall CPU utilization, as one would expect due to the minimal overhead. First come first serve avoids unnecessary context switching and other calculation times, thus maximizing the CPU utilization.
- Throughput:
    - Mean: roughly the same as for SJF, with a slightly higher mean in some cases. But overall, the two are even.
    - Median: Substantially lower than the mean, likely indicating many low values and a few very high values, this is as one would expect due to all the patrons getting their drinks in the order in which they are placed, all patrons are likely to receive their final order at roughly the same time, and initially, although many drinks are being made (as evident by the CPU utilization) no patron is complete.
    - Standard deviation: Highest overall standard deviation, likely due to the reasons listed for the median. This is as we would expect.
- Turnaround, Wait and Response times:
    - Mean and median: Both increase linearly with number of patrons and FCFS is fairly even with RR, however the median is slightly better for FCFS. Overall the medians are higher than the means, likely indicating a large portion of small values and a few larger values.
    - Standard deviation: lowest overall standard deviation, indicating a lack of starvation, as we would expect for FCFS.

Round Robin:

- CPU utilization: lowest overall, however still above 90% for larger number of patrons, this is as we would expect with a low context switch time, but as seen in the context switch tests, this would decrease with a larger context switch.
- Throughput:
    - Mean: overall lowest mean, but only by a small margin.
    - Median: Substantially lower than the mean, likely indicating many low values and a few very high values, this is as one would expect due to the processing of drinks mostly in the order of arrival (80%), except when they are particularly long preparation times, all patrons are likely to receive their final order at roughly the same time, and initially, although many drinks are being made (as evident by the high CPU utilization) no patron is complete.
    - Standard deviation: second highest overall, due to the same reasons listed above.
- Turnaround, Wait and Response times:
    - No clear improvement over FCFS, in some cases the mean or median is slightly higher or lower, however this is likely due to chance and may not follow the same pattern for an extensive number of tests.

Conclusions:

Algorithm behavior:

Overall, the algorithms behaved as expected, with perhaps lower performance than initially expected for large number of patrons. However, Round Robin and First come first served both preformed very similarly overall and as one would expect a fair algorithm to behave, with very predictable outcomes, overall low standard deviation and no evidence of starvation. Shortest Job first, overall, had the highest performance in many metrics, with the highest standard deviation in many of them as well. Once again this behavior is as we would expect for an unfair algorithm, and there was some evidence of starvation in response times.

Recommendation:

Overall, my recommendation would still be to use shortest job first scheduling due to the overall improvements in the mean and median values, despite the large standard deviations. In the context of an operating system, while a process may occasionally get no processing time for an excessive period, most processes will be extremely fast. And in the context of a barman, a person who orders a drink which is difficult to prepare likely expects to wait somewhat longer than a person ordering a shot, and thus, despite some having long wait times, one would hope that human logic would avoid too many bar fights.
Shortest Job First provides overall better performance averages despite occasionally unpredictable times and thus is the best option currently available to Sarah The Zombie Barman.

(and perhaps hire a second bartender who enjoys preparing cocktails?)